1. Labor

Az új környezet és a referencia típus

Általános információk

1. iMSC pontok

Az első laboron még nem szerezhető iMSc pont. A többi laboron szerezhető iMSc-s feladatok megoldásainak ***AUT portál***ra *.zip fájlba csomagoltan való feltöltési határideje az aktuális labortól számított három nap. Ha egy feladatban kérdések szerepelnek, a pontok csak akkor fogadhatók el, ha mellékletben egy **igényes** jegyzőkönyv is szerepel a kérdésekre vonatkozó válaszokkal. iMSc pont szerzésére bármely hallgató jogosult, aki az előtte lévő feladatokkal már végzett (laborvezető ellenőrzi a haladást).

2. Házi feladat

Házi feladat pontozási irányelvekről bővebben a „HF_IRANYELVEK.pdf" szól.

Kötelező feladatok

1. Emlékeztető, ismerkedés az új környezettel

Visual Studioban készíts egy új *Win32 Console Application* projektet *GettingStarted* néven, amely valósítsa meg az alábbiakat:

- *Olvasson be* egy egész számot (N), majd egy szöveget (S)
- N-t és S-t adja át egy *függvény*nek (F)
- F-ben S-t a standard kimenetre *írja ki* N-szer

2. Forráskód dekompozíció

Az előző feladat F függvényét valósítsd meg kétféleképpen:

1. Külön *printing.cpp* fájlban definiáld, majd abban a fájlban, amiben használod (*main.cpp*), *extern* kulcsszóval jelezd a fordítónak, hogy egy másik *.cpp-fájlban keresse a definíciót
2. Töröld az *extern*-t, helyette hozz létre egy *printing.h*-t
    a. csak a deklaráció szerepeljen benne, a definíció maradjon a *printing.cpp*-ben
    b. oldd meg, hogy többszöri include-olás esetén ne legyen többszörös deklaráció

3. Kódrészlet debuggolás

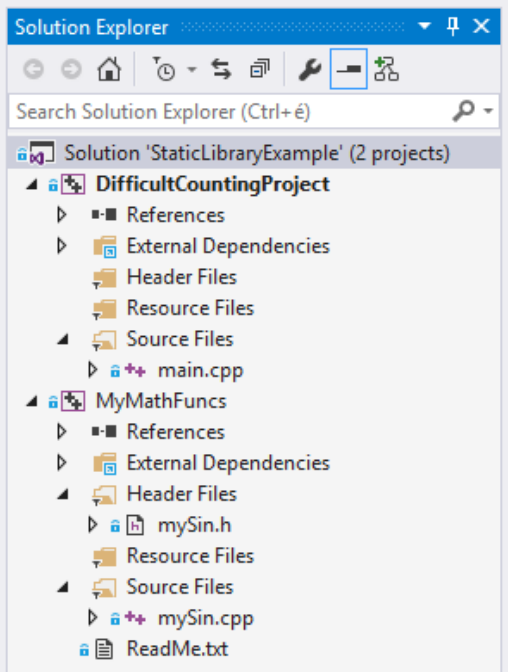
A Visual Studio eszközeit felhasználva futtasd utasításonként a következő kódrészleteket:

C- rossz	C-jó	C++

`#include<stdio.h>`
 	
`void f(int i)`
`{`
`  i=i+2;`
`}`
 	
`int main(void)`
`{`
`  int i=0;`
`  f(i);`
`  printf("%d\n",i);`
`}`
`/* A kimenet 0 */`
	`#include<stdio.h>`
	
	`void f(int* pi)`
	`{`
	`  (*pi)=(*pi)+2;`
	`}`
	
	`int main(void)`
	`{`
	`  int i=0;`
	`  f(&i);`
	`  printf("%d\n",i);`
	`}`
	`/* A kimenet 2 */`
		`#include<stdio.h>`
		
		`void f(int& i)`
		`{`
		`  i=i+2;`
		`}`
		
		`int main(void)`
		`{`
		`  int i=0;`
		`  f(i);`
		`  printf("%d\n",i);`
		`}`
		`/* A kimenet 2 */`

4. Statikus programkönyvtár készítése

1. Készíts egy *Blank Solution*-t StaticLibraryExample néven
2. Ezen belül készíts egy *Win32 Console Application* projektet MyMathFuncs néven
   - legyen static library



VS2017 alatt fontos változás, hogy **nem a Windows Console Application-t kell választani, hanem az "Application Wizard" projekttípust** a Visual C++ projekttípusok közül, az felhozza a formot, és ugyanúgy be lehet állítani a dolgokat, mint eddig.

3. készíts egy *mySin.h* és egy *mySin.cpp* állományt
4. implementálj egy *double awesomeSin(double x)* függvényt a sin(x) tetszőleges felhasználásával
5. projektre jobb klikk -> Build
   a. ezzel elkészült a StaticLibraryExample.lib, amit bármely más C++ programnál fel tudsz használni
   b. ez nem tartalmazza a *.h fájlokat

5. Külső statikus programkönyvtár használata
   1. Az előző projekt Solution-jébe (StaticLibraryExample) készíts még egy *Win32 Console Application* projektet DifficultCountingProject néven
   2. A projekt alatt: *References -> Add Reference ->pipáld ki a MyMathFuncs-t*
      a. így a linker már ismeri fogja az elkészített statikus programkönyvtárat
      b. azonban nincs hozzá deklaráció
      c. VS12: Projektre jobb klikk -> References… -> Add New Reference
   3. készíts egy *main.cpp* állományt
      a. enélkül egyrészt nem látszódna a következő lépés C/C++ menüpontja
   4. Projektre jobb klikk -> Properties -> Configuration Properties -> C/C++ -> General -> Additional Include Directories -> Tallózd be a másik projekt mappáját (ami tartalmazza a mySin.h-t)
      a. ez a lépés azért kell, hogy a program megtalálja a mySin.h-t
   5. írj példakódot a másik projektben felhasznált függvény segítségével
      a. ne felejtsd el include-olni a mySin.h-t a main.cpp-ben

Gyakorló feladatok

- Referencia helytelen/helyes használata a)-e)